Effat University
Department of Computer Science
Energy and Information Technology Research Center

LAB 4

Linear Regression & Its Variants

CS4082 – Machine Learning

Predicting Continuous Values with Scikit-Learn and Real-World Data

Prepared by: Dr. Naila Marir
Semester: Spring 2026

Contents

1 Lab Overview **3**
1.1 What You Will Learn . . . 3
1.2 Prerequisites . . . 3

2 Part 1: The Idea Behind Linear Regression **4**
2.1 The Mathematical Model . . . 4
2.2 What Does "Best Fit" Mean? . . . 4

3 Part 2: Loading a Real Dataset – California Housing **5**
3.1 Dataset at a Glance . . . 5
3.2 Step 1: Load and Explore . . . 5
3.3 Step 2: Understand the Features . . . 5
3.4 Step 3: Visualize Relationships . . . 5

4 Part 3: Simple Linear Regression (One Feature) **7**
4.1 Step 1: Prepare Data . . . 7
4.2 Step 2: Train and Visualize . . . 7

5 Part 4: Evaluating Regression Models **9**
5.1 The Four Key Metrics . . . 9
5.2 Step 1: Evaluate the Simple Model . . . 9
5.3 Step 2: Visualize Predictions vs. Actual . . . 9

6 Part 5: Multiple Linear Regression (All Features) **11**
6.1 Step 1: Train with All Features . . . 11
6.2 Step 2: Feature Importance – Which Features Matter Most? . . . 11

7 Part 6: Polynomial Regression **13**
7.1 The Key Idea . . . 13
7.2 Step 1: Polynomial on a Single Feature . . . 13
7.3 Step 2: Polynomial on All Features . . . 14

8 Part 7: Regularized Regression – Ridge, Lasso, and ElasticNet **15**
8.1 The Three Variants . . . 15
8.2 Step 1: Train All Three Variants . . . 15
8.3 Step 2: Compare Coefficients . . . 16

9 Part 8: The Alpha Tuning Experiment **17**
9.1 Step 1: Ridge Alpha Search . . . 17

10 Part 9: Final Model Comparison **19**
10.1 Step 1: Comprehensive Comparison . . . 19
10.2 Step 2: Visual Comparison . . . 20

11 Part 10: Apply to a New Dataset – Saudi Housing **21**
11.1 Step 1: Create the Saudi Housing Dataset . . . 21

12 Part 11: Summary and Key Takeaways **23**
12.1 Models Covered in This Lab . . . 23
12.2 Key Rules to Remember . . . 23
12.3 What is Coming Next . . . 23

13 Submission Requirements **24**
13.1 What to Submit 24
13.2 Grading Rubric 24

1 Lab Overview

In Labs 2 and 3, you worked with **classification** – predicting which *category* a sample belongs to (e.g., flower species, disease presence). In this lab, we shift to **regression** – predicting a *continuous number* (e.g., house price, temperature, salary).

You will use a real-world housing dataset to learn Simple Linear Regression, Multiple Linear Regression, Polynomial Regression, and Regularized Regression (Ridge, Lasso, ElasticNet) – all through scikit-learn.

Classification vs. Regression

- **Classification**: Output is a category. *"Is this email spam or not?"*
- **Regression**: Output is a number. *"What will this house sell for?"*

The scikit-learn workflow is identical: `fit` → `predict` → `evaluate`. Only the models and metrics change!

1.1 What You Will Learn

- The difference between classification and regression
- Simple Linear Regression with one feature
- Multiple Linear Regression with many features
- Regression evaluation metrics: MSE, RMSE, MAE, R^2
- Polynomial Regression for non-linear relationships
- Regularization: Ridge (L_2), Lasso (L_1), and ElasticNet
- Comparing multiple regression models on real data

1.2 Prerequisites

- Completion of Labs 2 and 3 (scikit-learn basics, preprocessing, pipelines)
- Basic understanding of linear equations ($y = mx + b$)
- Google Colab account (recommended) or local Python 3.8+ installation

Looking Ahead

In this lab you will use scikit-learn's built-in regression models as powerful tools. In **Lab 5**, you will implement Linear Regression *from scratch* using gradient descent – so you understand exactly what happens inside the black box!

2 Part 1: The Idea Behind Linear Regression

Linear Regression finds the best straight line (or hyperplane) that relates input features to a continuous output.

2.1 The Mathematical Model

For a single feature, the model is simply:

$$\hat{y} = w_1 x + w_0$$

where w_1 is the **slope** (weight) and w_0 is the **intercept** (bias).

For multiple features, this generalizes to:

$$\hat{y} = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_n x_n$$

The goal of training is to find the weights $w_0, w_1, \ldots, w_n$ that **minimize the error** between predictions $\hat{y}$ and actual values y.

2.2 What Does "Best Fit" Mean?

Scikit-learn's `LinearRegression` minimizes the **Ordinary Least Squares (OLS)** objective:

$$\text{Cost} = \sum_{i=1}^{m} (y_i - \hat{y}_i)^2$$

This is the sum of squared differences between each actual value and its prediction. The line that makes this sum as small as possible is the "best fit."

Why Squared Errors?

Squaring the errors does two things: (1) it makes all errors positive (so positive and negative errors don't cancel out), and (2) it penalizes large errors more heavily than small ones, pushing the model to avoid big mistakes.

3 Part 2: Loading a Real Dataset – California Housing

We will use the **California Housing dataset**, a real dataset built into scikit-learn. It contains information about housing districts in California collected from the 1990 U.S. census. The goal is to predict the **median house value** for each district.

3.1 Dataset at a Glance

Property	Details
Samples	20,640 districts
Features	8 numeric attributes
Target	Median house value (in $100,000s)
Task	Predict house value from district characteristics

3.2 Step 1: Load and Explore

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing

# Load the dataset
housing = fetch_california_housing()
df = pd.DataFrame(housing.data, columns=housing.feature_names)
df['MedHouseVal'] = housing.target  # target in $100,000s

print(f'Dataset shape: {df.shape}')
print(f'\nFeature names:\n{housing.feature_names}')
print(f'\nFirst 5 rows:')
print(df.head())
print(f'\nBasic statistics:')
print(df.describe().round(3))
```

3.3 Step 2: Understand the Features

Feature	Description	Unit
MedInc	Median income of district	$10,000s
HouseAge	Median house age	Years
AveRooms	Average rooms per household	Count
AveBedrms	Average bedrooms per household	Count
Population	District population	People
AveOccup	Average household members	People
Latitude	District latitude	Degrees
Longitude	District longitude	Degrees
MedHouseVal	Median house value (**target**)	$100,000s

Table 1: California Housing dataset features.

3.4 Step 3: Visualize Relationships

```
fig, axes = plt.subplots(2, 3, figsize=(16, 10))
features_to_plot = ['MedInc', 'HouseAge', 'AveRooms',
                    'Population', 'Latitude', 'Longitude']

for ax, feat in zip(axes.flatten(), features_to_plot):
    ax.scatter(df[feat], df['MedHouseVal'],
               alpha=0.1, s=5, color='steelblue')
    ax.set_xlabel(feat)
    ax.set_ylabel('MedHouseVal ($100k)')
    ax.set_title(f'{feat} vs House Value')

plt.tight_layout()
plt.show()
```

What to Notice in the Plots

MedInc (median income) shows a clear upward trend – higher-income districts have more expensive houses. This will likely be the most predictive feature. Some relationships are non-linear (e.g., Latitude), which is why we will try Polynomial Regression later.

Task 1: Explore the Dataset

- Use `df.corr()['MedHouseVal'].sort_values()` to rank features by their correlation with the target. Which feature has the strongest correlation?
- Plot the distribution of `MedHouseVal` using `plt.hist(df['MedHouseVal'], bins=50)`. Is the target normally distributed? Do you notice any cap at $5.0 (i.e., $500,000)?
- How many samples are in the dataset? Is this enough for training a good model?

4 Part 3: Simple Linear Regression (One Feature)

Let's start with the simplest case: predicting house value using *only* median income. This lets us visualize the regression line in 2D.

4.1 Step 1: Prepare Data

```
from sklearn.model_selection import train_test_split

# Use only MedInc as the feature
X_simple = df[['MedInc']].values
y = df['MedHouseVal'].values

# Split: 80% train, 20% test
X_train, X_test, y_train, y_test = train_test_split(
    X_simple, y, test_size=0.2, random_state=42)

print(f'Training samples: {X_train.shape[0]}')
print(f'Testing samples:  {X_test.shape[0]}')
```

4.2 Step 2: Train and Visualize

```
from sklearn.linear_model import LinearRegression

# Train
model_simple = LinearRegression()
model_simple.fit(X_train, y_train)

# Print the learned parameters
print(f'Slope (w1):     {model_simple.coef_[0]:.4f}')
print(f'Intercept (w0): {model_simple.intercept_:.4f}')
print(f'\nEquation: y = {model_simple.coef_[0]:.4f} * MedInc '
      f'+ ({model_simple.intercept_:.4f})')

# Visualize: scatter + regression line
plt.figure(figsize=(10, 6))
plt.scatter(X_test, y_test, alpha=0.2, s=10,
            color='steelblue', label='Actual')

# Draw the regression line
x_line = np.linspace(X_test.min(), X_test.max(), 100)
y_line = model_simple.predict(x_line.reshape(-1, 1))
plt.plot(x_line, y_line, color='red', linewidth=2,
         label='Regression Line')

plt.xlabel('Median Income ($10,000s)')
plt.ylabel('Median House Value ($100,000s)')
plt.title('Simple Linear Regression: Income vs House Value')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

Reading the Equation

If the model learns $\hat{y} = 0.42 \times \text{MedInc} + 0.45$, this means: for every \$10,000 increase in median income, house value increases by about \$42,000. The intercept (\$45,000) is the predicted value when income is zero – not physically meaningful, but mathematically necessary.

Task 2: Interpret the Simple Model

- What are the slope and intercept your model learned? Write the equation.
- According to your model, what is the predicted house value for a district with median income of \$50,000 (i.e., `MedInc = 5.0`)? Calculate by hand and verify with `model_simple.predict([[5.0]])`.
- Try training a simple model using `HouseAge` instead of `MedInc`. Is it better or worse? Why?

5 Part 4: Evaluating Regression Models

In classification, we used accuracy. For regression, we need different metrics since our predictions are continuous numbers, not categories.

5.1 The Four Key Metrics

Metric	Formula	Interpretation	Best Value
MSE	$\frac{1}{m}\sum(y_i - \hat{y}_i)^2$	Average squared error	0
RMSE	$\sqrt{\text{MSE}}$	Error in original units	0
MAE	$\frac{1}{m}\sum\lvert y_i - \hat{y}_i\rvert$	Average absolute error	0
R^2	$1 - \frac{\sum(y_i-\hat{y}_i)^2}{\sum(y_i-\bar{y})^2}$	Variance explained	1.0

Table 2: Regression evaluation metrics.

Understanding R^2 (R-squared)

$R^2 = 1.0$ means the model explains 100% of the variance (perfect predictions). $R^2 = 0$ means the model is no better than simply predicting the mean every time. R^2 can even be negative if the model is worse than the mean baseline!

5.2 Step 1: Evaluate the Simple Model

```
from sklearn.metrics import mean_squared_error, mean_absolute_error,
    r2_score

# Predictions
y_pred_simple = model_simple.predict(X_test)

# Calculate metrics
mse  = mean_squared_error(y_test, y_pred_simple)
rmse = np.sqrt(mse)
mae  = mean_absolute_error(y_test, y_pred_simple)
r2   = r2_score(y_test, y_pred_simple)

print('=== Simple Linear Regression (MedInc only) ===')
print(f'MSE:  {mse:.4f}')
print(f'RMSE: {rmse:.4f}  (error in $100,000s)')
print(f'MAE:  {mae:.4f}  (error in $100,000s)')
print(f'R^2:  {r2:.4f}  ({r2*100:.1f}% variance explained)')
```

5.3 Step 2: Visualize Predictions vs. Actual

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Plot 1: Predicted vs Actual
axes[0].scatter(y_test, y_pred_simple, alpha=0.2, s=10,
                color='steelblue')
axes[0].plot([0, 5], [0, 5], 'r--', linewidth=2,
             label='Perfect Prediction')
```

```
axes[0].set_xlabel('Actual House Value ($100k)')
axes[0].set_ylabel('Predicted House Value ($100k)')
axes[0].set_title('Predicted vs Actual')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Plot 2: Residuals (errors)
residuals = y_test - y_pred_simple
axes[1].scatter(y_pred_simple, residuals, alpha=0.2, s=10,
                color='coral')
axes[1].axhline(y=0, color='black', linewidth=1)
axes[1].set_xlabel('Predicted Value')
axes[1].set_ylabel('Residual (Actual - Predicted)')
axes[1].set_title('Residual Plot')
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

Reading Residual Plots

In a good model, residuals should be randomly scattered around zero with no visible pattern. If you see a curve or funnel shape, it means the model is systematically wrong in certain regions – a sign that a non-linear model might work better.

Task 3: Understand the Metrics

- Convert the RMSE to actual dollars. If RMSE ≈ 0.74, what is the average prediction error in dollars?
- Look at the "Predicted vs Actual" plot. Where does the model struggle most – at low values, high values, or both?
- What does the R^2 value tell you? Is a single feature enough to predict house prices well?

6 Part 5: Multiple Linear Regression (All Features)

A single feature captures only part of the story. Let's use *all 8 features* and see how much the model improves.

6.1 Step 1: Train with All Features

```
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

# Use all features
X_all = df.drop(columns=['MedHouseVal']).values
y = df['MedHouseVal'].values

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X_all, y, test_size=0.2, random_state=42)

# Build pipeline with scaling (good practice!)
pipe_lr = Pipeline([
    ('scaler', StandardScaler()),
    ('lr', LinearRegression())
])

pipe_lr.fit(X_train, y_train)
y_pred_multi = pipe_lr.predict(X_test)

# Evaluate
r2_multi = r2_score(y_test, y_pred_multi)
rmse_multi = np.sqrt(mean_squared_error(y_test, y_pred_multi))

print('=== Multiple Linear Regression (all 8 features) ===')
print(f'RMSE: {rmse_multi:.4f}')
print(f'R^2:  {r2_multi:.4f}  ({r2_multi*100:.1f}% variance explained)
    ')
print(f'\nImprovement over simple model:')
print(f'  R^2: {r2:.4f} -> {r2_multi:.4f} '
      f'(+{(r2_multi - r2)*100:.1f}%)')
```

6.2 Step 2: Feature Importance – Which Features Matter Most?

```
# Extract the learned coefficients
lr_model = pipe_lr.named_steps['lr']
feature_names = housing.feature_names

# Since we scaled, coefficients are comparable
coef_df = pd.DataFrame({
    'Feature': feature_names,
    'Coefficient': lr_model.coef_
}).sort_values('Coefficient', key=abs, ascending=False)

print('Feature Importance (by |coefficient|):')
print(coef_df.to_string(index=False))

# Visualize
plt.figure(figsize=(10, 5))
```

```
colors = ['steelblue' if c > 0 else 'coral'
          for c in coef_df['Coefficient']]
plt.barh(coef_df['Feature'], coef_df['Coefficient'], color=colors)
plt.xlabel('Coefficient Value (after scaling)')
plt.title('Feature Importance in Multiple Linear Regression')
plt.axvline(x=0, color='black', linewidth=0.5)
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()
```

Interpreting Scaled Coefficients

Because we used `StandardScaler`, all features have the same scale. A larger absolute coefficient means the feature has more influence on the prediction. Positive coefficients increase house value; negative coefficients decrease it. For example, `MedInc` having the largest positive coefficient confirms that income is the strongest predictor.

Task 4: Analyze Multiple Regression

- List the top 3 most important features and the bottom 3. Does this make intuitive sense?
- Why does `Latitude` have a negative coefficient? What does this mean geographically for California?
- Create the "Predicted vs Actual" plot for the multiple regression model. Compare it visually with the simple model. Is it closer to the diagonal?

7 Part 6: Polynomial Regression

Some relationships between features and target are not linear – they curve. Polynomial Regression handles this by creating new features that are powers of the originals (e.g., x^2, x^3), then fitting a linear model on the expanded feature set.

7.1 The Key Idea

For a single feature x, Polynomial Regression of degree 2 transforms:

$$\hat{y} = w_0 + w_1 x \quad \longrightarrow \quad \hat{y} = w_0 + w_1 x + w_2 x^2$$

The model is still *linear in the weights* (it just finds w_0, w_1, w_2), but the relationship with x is now a curve.

7.2 Step 1: Polynomial on a Single Feature

Let's see the effect on the MedInc feature:

```
from sklearn.preprocessing import PolynomialFeatures

# Use only MedInc
X_simple = df[['MedInc']].values
X_train_s, X_test_s, y_train, y_test = train_test_split(
    X_simple, y, test_size=0.2, random_state=42)

# Try degrees 1, 2, and 3
results = {}
plt.figure(figsize=(15, 4))

for i, degree in enumerate([1, 2, 3]):
    pipe_poly = Pipeline([
        ('poly', PolynomialFeatures(degree=degree,
                                    include_bias=False)),
        ('lr', LinearRegression())
    ])
    pipe_poly.fit(X_train_s, y_train)
    y_pred = pipe_poly.predict(X_test_s)

    r2_val = r2_score(y_test, y_pred)
    rmse_val = np.sqrt(mean_squared_error(y_test, y_pred))
    results[degree] = {'R2': r2_val, 'RMSE': rmse_val}

    # Plot
    ax = plt.subplot(1, 3, i + 1)
    ax.scatter(X_test_s, y_test, alpha=0.1, s=5,
               color='steelblue')

    x_plot = np.linspace(X_test_s.min(), X_test_s.max(),
                         300).reshape(-1, 1)
    y_plot = pipe_poly.predict(x_plot)
    ax.plot(x_plot, y_plot, color='red', linewidth=2)
    ax.set_title(f'Degree {degree} (R$^2$={r2_val:.3f})')
    ax.set_xlabel('MedInc')
    ax.set_ylabel('MedHouseVal')

plt.tight_layout()
plt.show()
```

```
# Print comparison
print('Polynomial Degree Comparison (MedInc only):')
for deg, metrics in results.items():
    print(f"  Degree {deg}: R^2={metrics['R2']:.4f}, "
          f"RMSE={metrics['RMSE']:.4f}")
```

7.3 Step 2: Polynomial on All Features

```
from sklearn.preprocessing import StandardScaler

# Use all features with degree 2
X_train, X_test, y_train, y_test = train_test_split(
    X_all, y, test_size=0.2, random_state=42)

pipe_poly_all = Pipeline([
    ('scaler', StandardScaler()),
    ('poly', PolynomialFeatures(degree=2, include_bias=False)),
    ('lr', LinearRegression())
])

pipe_poly_all.fit(X_train, y_train)
y_pred_poly = pipe_poly_all.predict(X_test)

r2_poly = r2_score(y_test, y_pred_poly)
rmse_poly = np.sqrt(mean_squared_error(y_test, y_pred_poly))

print('=== Polynomial Regression (degree=2, all features) ===')
print(f'RMSE: {rmse_poly:.4f}')
print(f'R^2:  {r2_poly:.4f}  ({r2_poly*100:.1f}% variance explained)')

# How many features now?
poly_features = pipe_poly_all.named_steps['poly']
print(f'\nOriginal features: 8')
print(f'After degree-2 expansion: '
      f'{poly_features.n_output_features_}')
```

The Feature Explosion Problem

With 8 original features and degree 2, `PolynomialFeatures` creates *all* pairwise combinations plus squares: $\binom{8+2}{2} - 1 = 44$ features! At degree 3, this explodes to 164 features. More features means more risk of **overfitting** – the model memorizes training data instead of learning patterns. This is where regularization comes in.

Task 5: Explore Polynomial Regression

- Look at the three plots for degrees 1, 2, and 3. Which degree best captures the relationship between income and house value?
- Try degree 5 on the single-feature model. Does R^2 keep improving, or does it start to behave strangely at the edges? This is overfitting!
- How many features does degree 3 create from 8 original features? Use `PolynomialFeatures(degree=3).fit_transform(X_train).shape` to check.

8 Part 7: Regularized Regression – Ridge, Lasso, and Elastic-Net

When models have many features (especially after polynomial expansion), they can overfit. **Regularization** adds a penalty to the cost function that discourages large weights, keeping the model simpler and more generalizable.

8.1 The Three Variants

Model	Penalty	Effect	Best For
Ridge (L_2)	$\alpha \sum w_i^2$	Shrinks all weights	Many useful features
Lasso (L_1)	$\alpha \sum \lvert w_i \rvert$	Some weights $\to 0$	Feature selection
ElasticNet	$\alpha_1 \sum \lvert w_i \rvert + \alpha_2 \sum w_i^2$	Combination of both	Correlated features

Table 3: Regularization methods comparison.

The parameter α controls the regularization strength: $\alpha = 0$ gives standard Linear Regression; larger α values mean stronger regularization (simpler model).

8.2 Step 1: Train All Three Variants

```
from sklearn.linear_model import Ridge, Lasso, ElasticNet

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X_all, y, test_size=0.2, random_state=42)

# Define models with pipelines
models = {
    'LinearRegression': Pipeline([
        ('scaler', StandardScaler()),
        ('model', LinearRegression())
    ]),
    'Ridge (alpha=1.0)': Pipeline([
        ('scaler', StandardScaler()),
        ('model', Ridge(alpha=1.0, random_state=42))
    ]),
    'Lasso (alpha=0.01)': Pipeline([
        ('scaler', StandardScaler()),
        ('model', Lasso(alpha=0.01, random_state=42))
    ]),
    'ElasticNet (alpha=0.01)': Pipeline([
        ('scaler', StandardScaler()),
        ('model', ElasticNet(alpha=0.01, l1_ratio=0.5,
                             random_state=42))
    ]),
}

# Train and evaluate each model
print(f'{"Model":<28} {"RMSE":>8} {"MAE":>8} {"R^2":>8}')
print('-' * 56)

results_dict = {}
```

```
for name, pipe in models.items():
    pipe.fit(X_train, y_train)
    y_pred = pipe.predict(X_test)

    rmse_val = np.sqrt(mean_squared_error(y_test, y_pred))
    mae_val  = mean_absolute_error(y_test, y_pred)
    r2_val   = r2_score(y_test, y_pred)

    results_dict[name] = {
        'RMSE': rmse_val, 'MAE': mae_val, 'R2': r2_val}
    print(f'{name:<28} {rmse_val:>8.4f} {mae_val:>8.4f} '
          f'{r2_val:>8.4f}')
```

8.3 Step 2: Compare Coefficients

The real power of regularization shows in the coefficients:

```
fig, axes = plt.subplots(1, 3, figsize=(16, 5))
model_names = ['Ridge (alpha=1.0)',
               'Lasso (alpha=0.01)',
               'ElasticNet (alpha=0.01)']
colors_list = ['steelblue', 'coral', 'seagreen']

for ax, name, clr in zip(axes, model_names, colors_list):
    coefs = models[name].named_steps['model'].coef_
    ax.barh(housing.feature_names, coefs, color=clr)
    ax.set_title(name)
    ax.axvline(x=0, color='black', linewidth=0.5)
    ax.set_xlabel('Coefficient')

plt.tight_layout()
plt.show()

# Check which features Lasso set to zero
lasso_coefs = models['Lasso (alpha=0.01)'].named_steps['model'].coef_
print('\nLasso coefficients:')
for feat, coef in zip(housing.feature_names, lasso_coefs):
    status = '' if abs(coef) > 0.001 else ' <-- REMOVED'
    print(f'  {feat:12s}: {coef:>8.4f}{status}')
```

Lasso as Feature Selection

Notice that Lasso drives some coefficients to **exactly zero**. This effectively removes those features from the model! Lasso simultaneously selects the most important features and trains the model – very useful when you have hundreds of features and want to identify the key ones.

Task 6: Explore Regularization

- Try different α values for Ridge: 0.01, 0.1, 1.0, 10.0, 100.0. Plot R^2 vs α. What happens with very large α?
- For Lasso, try `alpha=0.1`. How many features are removed (coefficient = 0)? Which ones?
- In the ElasticNet, change `l1_ratio` from 0.0 to 1.0. At `l1_ratio=0.0`, what model does it become? At `l1_ratio=1.0`?

9 Part 8: The Alpha Tuning Experiment

Choosing the right α is critical. Let's systematically search for the best value using a loop.

9.1 Step 1: Ridge Alpha Search

```
alphas = [0.001, 0.01, 0.1, 0.5, 1.0, 5.0, 10.0,
          50.0, 100.0, 500.0]

train_scores = []
test_scores = []

for alpha in alphas:
    pipe = Pipeline([
        ('scaler', StandardScaler()),
        ('ridge', Ridge(alpha=alpha))
    ])
    pipe.fit(X_train, y_train)

    train_scores.append(r2_score(y_train,
                                 pipe.predict(X_train)))
    test_scores.append(r2_score(y_test,
                                pipe.predict(X_test)))

# Plot
plt.figure(figsize=(10, 5))
plt.semilogx(alphas, train_scores, 'o-',
             label='Train R^2', color='steelblue')
plt.semilogx(alphas, test_scores, 's-',
             label='Test R^2', color='coral')
plt.xlabel('Alpha (log scale)')
plt.ylabel('R^2 Score')
plt.title('Ridge Regression: Effect of Alpha')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

# Best alpha
best_idx = np.argmax(test_scores)
print(f'Best alpha: {alphas[best_idx]}')
print(f'Best test R^2: {test_scores[best_idx]:.4f}')
```

The Bias-Variance Tradeoff

- **Small** α: Low regularization → complex model → low bias, high variance (risk of overfitting).
- **Large** α: Strong regularization → simple model → high bias, low variance (risk of underfitting).
- **Best** α: The sweet spot where the test score is maximized – the model is complex enough to capture patterns but simple enough to generalize.

Task 7: Tune Lasso Alpha

- Repeat the alpha search experiment for **Lasso** using alphas: `[0.0001, 0.001, 0.005, 0.01, 0.05, 0.1, 0.5, 1.0]`.

- Plot train and test R^2 vs alpha, just like the Ridge plot above.
- For each alpha, count how many features Lasso keeps (coefficient $\neq 0$). At what alpha does Lasso start removing features aggressively?
- Which alpha gives the best test R^2 for Lasso?

10 Part 9: Final Model Comparison

Let's bring everything together and compare all the models we have built in this lab.

10.1 Step 1: Comprehensive Comparison

```
# Rebuild all models on the same split
X_train, X_test, y_train, y_test = train_test_split(
    X_all, y, test_size=0.2, random_state=42)

all_models = {
    'Simple LR (MedInc)': Pipeline([
        ('lr', LinearRegression())
    ]),
    'Multiple LR (all)': Pipeline([
        ('scaler', StandardScaler()),
        ('lr', LinearRegression())
    ]),
    'Polynomial (deg=2)': Pipeline([
        ('scaler', StandardScaler()),
        ('poly', PolynomialFeatures(degree=2,
                                    include_bias=False)),
        ('lr', LinearRegression())
    ]),
    'Ridge': Pipeline([
        ('scaler', StandardScaler()),
        ('model', Ridge(alpha=1.0))
    ]),
    'Lasso': Pipeline([
        ('scaler', StandardScaler()),
        ('model', Lasso(alpha=0.01))
    ]),
    'ElasticNet': Pipeline([
        ('scaler', StandardScaler()),
        ('model', ElasticNet(alpha=0.01, l1_ratio=0.5))
    ]),
}

# Evaluate
print(f'{"Model":<25} {"RMSE":>8} {"MAE":>8} '
      f'{"R^2":>8}  {"# Features":>10}')
print('=' * 67)

comparison_data = []
for name, pipe in all_models.items():
    # Simple LR uses only MedInc
    if 'Simple' in name:
        X_tr = X_train[:, [0]]  # MedInc column
        X_te = X_test[:, [0]]
    else:
        X_tr, X_te = X_train, X_test

    pipe.fit(X_tr, y_train)
    y_pred = pipe.predict(X_te)

    rmse_v = np.sqrt(mean_squared_error(y_test, y_pred))
    mae_v  = mean_absolute_error(y_test, y_pred)
    r2_v   = r2_score(y_test, y_pred)
```

```
    # Count effective features
    if 'poly' in pipe.named_steps:
        n_feat = pipe.named_steps['poly'].n_output_features_
    elif 'Simple' in name:
        n_feat = 1
    else:
        n_feat = X_train.shape[1]

    comparison_data.append({
        'Model': name, 'RMSE': rmse_v,
        'MAE': mae_v, 'R2': r2_v, 'Features': n_feat})
    print(f'{name:<25} {rmse_v:>8.4f} {mae_v:>8.4f} '
          f'{r2_v:>8.4f}  {n_feat:>10}')
```

10.2 Step 2: Visual Comparison

```
comp_df = pd.DataFrame(comparison_data)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# R^2 comparison
axes[0].barh(comp_df['Model'], comp_df['R2'], color='steelblue')
axes[0].set_xlabel('R^2 Score')
axes[0].set_title('Model Comparison: R^2 (higher is better)')
axes[0].set_xlim(0, 1)

# RMSE comparison
axes[1].barh(comp_df['Model'], comp_df['RMSE'], color='coral')
axes[1].set_xlabel('RMSE')
axes[1].set_title('Model Comparison: RMSE (lower is better)')

plt.tight_layout()
plt.show()
```

Task 8: Draw Conclusions

- Which model achieved the best R^2 on the test set? Which had the lowest RMSE?
- The Polynomial (degree 2) model uses 44 features. Is the improvement in R^2 worth the added complexity? Justify your answer.
- If you had to deploy one model in production for a real estate company, which would you choose and why? Consider accuracy, interpretability, and simplicity.
- Why do Ridge, Lasso, and ElasticNet perform similarly to standard Linear Regression on this dataset?

11 Part 10: Apply to a New Dataset – Saudi Housing

To solidify your skills, let's apply the complete regression workflow to a different dataset. We will create a synthetic Saudi housing dataset and you will run the full analysis.

11.1 Step 1: Create the Saudi Housing Dataset

```
import pandas as pd
import numpy as np

np.random.seed(42)
n = 500

cities = np.random.choice(
    ['Jeddah', 'Riyadh', 'Dammam', 'Makkah', 'Madinah'],
    n, p=[0.30, 0.30, 0.15, 0.15, 0.10])

# Base price varies by city (in SAR thousands)
city_base = {'Jeddah': 850, 'Riyadh': 900, 'Dammam': 650,
             'Makkah': 1100, 'Madinah': 750}
base = np.array([city_base[c] for c in cities], dtype=float)

area = np.round(np.random.uniform(80, 400, n), 1)
bedrooms = np.random.randint(1, 7, n)
bathrooms = np.random.randint(1, 5, n)
age = np.random.randint(0, 40, n)
floor = np.random.randint(1, 15, n)
has_parking = np.random.choice([0, 1], n, p=[0.3, 0.7])

# Generate price with realistic relationships
price = (base
         + 3.5 * area
         + 80 * bedrooms
         + 50 * bathrooms
         - 12 * age
         + 15 * floor
         + 100 * has_parking
         + np.random.normal(0, 150, n))  # noise
price = np.round(np.maximum(price, 200), 1)

df_saudi = pd.DataFrame({
    'city': cities,
    'area_sqm': area,
    'bedrooms': bedrooms,
    'bathrooms': bathrooms,
    'age_years': age,
    'floor': floor,
    'has_parking': has_parking,
    'price_kSAR': price  # price in thousands of SAR
})

df_saudi.to_csv('saudi_housing.csv', index=False)
print(f'Dataset shape: {df_saudi.shape}')
print(df_saudi.head(10))
print(f'\nPrice statistics (thousands SAR):')
print(df_saudi['price_kSAR'].describe().round(1))
```

Task 9: Full Saudi Housing Analysis (Challenge)

Apply everything you learned in this lab to the `saudi_housing.csv` dataset:

1. **Explore**: Use `df.describe()`, `df.corr()`, and scatter plots to understand the data. Which feature correlates most with price?
2. **Preprocess**: Encode the `city` column using One-Hot encoding (from Lab 3). Scale the numeric features.
3. **Simple model**: Train a Linear Regression using only `area_sqm`. Plot the regression line and report R^2.
4. **Multiple model**: Train with all features. How much does R^2 improve?
5. **Regularized models**: Train Ridge, Lasso, and ElasticNet. Compare their performance.
6. **Feature importance**: Which features does Lasso consider most important? Does this match your intuition about Saudi housing prices?
7. **Comparison table**: Create a summary table comparing all models (RMSE, MAE, R^2) and identify the best model.

This task combines skills from Labs 3 and 4. Use pipelines throughout!

12 Part 11: Summary and Key Takeaways

12.1 Models Covered in This Lab

Model	Key Idea	Strengths	When to Use
Linear Regression	Minimize squared errors	Simple, interpretable	Linear relationships
Polynomial	Add power terms	Captures curves	Non-linear data
Ridge (L_2)	Penalize large weights	Stable predictions	Many features
Lasso (L_1)	Force weights to zero	Feature selection	Sparse models
ElasticNet	$L_1 + L_2$	Best of both	Correlated features

Table 4: Summary of regression models.

12.2 Key Rules to Remember

1. **Always visualize** your data before modeling – scatter plots reveal relationships and outliers.
2. **Start simple**: Try Linear Regression first, then add complexity only if needed.
3. **Evaluate properly**: Use R^2 to measure how much variance is explained and RMSE for error in original units.
4. **Regularize when needed**: Ridge and Lasso prevent overfitting, especially with many features or polynomial expansion.
5. **Use Lasso for feature selection**: When you need to know which features matter most.
6. **Check residuals**: Random residuals = good model; patterns in residuals = model is missing something.

12.3 What is Coming Next

In **Lab 5**, you will implement Linear Regression **from scratch** using gradient descent. You will code the cost function, compute gradients manually, and watch the model learn step by step. This will give you deep understanding of what scikit-learn does behind the scenes!

13 Submission Requirements

13.1 What to Submit

1. Your completed Colab notebook (`.ipynb`) with all code cells executed and outputs visible.
2. A short report (5–7 sentences) comparing the regression models you trained. Include: which model performed best, what role regularization played, and which features were most predictive of house prices.

13.2 Grading Rubric

Criterion	**Points**	**Weight**
Data exploration and visualization (Tasks 1–2)	10	10%
Simple and Multiple Linear Regression (Tasks 3–4)	20	20%
Regression metrics and interpretation (Task 3)	10	10%
Polynomial Regression (Task 5)	15	15%
Regularized models and alpha tuning (Tasks 6–7)	15	15%
Final model comparison (Task 8)	10	10%
Saudi Housing challenge (Task 9)	10	10%
Written report and overall quality	10	10%
Total	**100**	**100%**

Table 5: Lab 4 grading rubric.

Final Tip

The models you used today are scikit-learn "black boxes" – you called `fit()` and they just worked. In Lab 5, you will **open the box** and code gradient descent yourself. Understanding the math behind Linear Regression is what separates an ML practitioner from someone who just runs code!